

Arquitectura técnica — MisFinanzas

Visión general

MisFinanzas es un monolito web de archivo único deliberadamente simple. Todo el backend vive en app.py y todo el frontend en static/index.html. No hay build steps, no hay node_modules, no hay frameworks de frontend con bundle. La filosofía es: menos capas = menos cosas que romper.

```
Browser
├─ GET /                → static/index.html  (React via CDN + Babel standalone)
├─ GET /api/*           → FastAPI (app.py)
│   └─ SQLAlchemy ORM
│       └─ SQLite (dev) | PostgreSQL (producción)
```

Stack

Capa	Tecnología	Por qué
HTTP server	FastAPI + Uvicorn	Async, tipado, validación automática con Pydantic
ORM	SQLAlchemy 2.x	Agnóstico a la DB — mismo código en SQLite y PostgreSQL
Auth	JWT (python-jose) + bcrypt	Stateless; tokens con expiración de 30 días
Rate limiting	slowapi	Anti fuerza bruta en endpoints de auth
IA	Google Gemini 2.0 Flash	API gratuita, multimodal, contexto largo
Frontend	React 18 (CDN) + Tailwind CSS (CDN) +	Sin build step; funciona directo del browser
Tasas	ve.dolarapi.com + Binance P2P	Dos fuentes independientes para redundancia

Modelos de datos

```
users
├─ id (PK)
├─ username (unique)
├─ hashed_password
└─ created_at

transactions
├─ id (PK)
├─ user_id (FK → users)
├─ date
├─ description
├─ category
└─ type ('ingreso' | 'gasto')
```

```

├─ amount      (monto original en la moneda nativa)
├─ currency    ('VES' | 'USD' | 'USDT')
├─ payment_method
├─ exchange_rate_bcv
├─ exchange_rate_binance
├─ amount_ves  (siempre calculado)
├─ amount_usd  (opcional)
├─ amount_usdt (opcional)
├─ notes
└─ tags        (CSV libre, ej: "vacaciones,trabajo")

```

goals

```

├─ id (PK)
├─ user_id (FK)
├─ name
├─ target_usdt
├─ notes
└─ created_at

```

budget_alerts

```

├─ id (PK)
├─ user_id (FK)
├─ category
├─ limit_ves
└─ active

```

rate_history

```

├─ id (PK)
├─ date
├─ bcv
└─ binance_p2p

```

Invariante de conversión

Toda transacción siempre tiene `amount_ves`. Las otras representaciones (`amount_usd`, `amount_usdt`) se calculan en el momento de creación/edición y se almacenan desnormalizadas para evitar recálculos. El cálculo:

```

USD  → amount_ves = amount × exchange_rate_bcv
USDT → amount_ves = amount × exchange_rate_binance
VES  → amount_ves = amount (identidad)

```

Endpoints de la API

Auth

Método	Path	Auth	Descripción
--------	------	------	-------------

POST	/api/auth/register	No	Registro (5/min)
POST	/api/auth/login	No	Login (10/min)
GET	/api/auth/me	Sí	Usuario actual

Transacciones

Método	Path	Descripción
GET	/api/transactions?skip=0&limit=500	Lista paginada
POST	/api/transactions	Crear
PUT	/api/transactions/{id}	Editar + recalcular montos
DELETE	/api/transactions/{id}	Eliminar
GET	/api/export/csv	Exportar todo a CSV (con BOM para Excel)

Resumen y reportes

Método	Path	Descripción
GET	/api/summary	Totales + alertas de presupuesto activas
GET	/api/monthly-flow	Flujo de los últimos 6 meses

Tasas de cambio

Método	Path	Descripción
GET	/api/rates	BCV + Binance P2P en tiempo real (guarda snapshot)
GET	/api/rates/history	Últimas 30 lecturas guardadas

Metas de ahorro

Método	Path	Descripción
GET	/api/goals	Lista de metas
POST	/api/goals	Crear meta
DELETE	/api/goals/{id}	Eliminar meta

Presupuestos

Método	Path	Descripción
GET	/api/budget-alerts	Lista de presupuestos
GET	/api/budget-alerts/check	Estado actual del mes (% gastado)
POST	/api/budget-alerts	Crear o actualizar presupuesto por categoría
DELETE	/api/budget-alerts/{id}	Eliminar

IA

Método	Path	Descripción
--------	------	-------------

GET	/api/ai/status	¿Hay API key configurada?
POST	/api/ai/key	Guardar API key de Gemini
POST	/api/ai/chat	Chat con contexto financiero del usuario

Compatibilidad SQLite / PostgreSQL

La función `month_label(col)` abstrae la diferencia de dialecto SQL:

```
def month_label(col):
    if _is_sqlite:
        return func.strftime("%Y-%m", col)    # SQLite
    return func.to_char(col, "YYYY-MM")      # PostgreSQL
```

Cualquier query que filtre o agrupe por mes usa esta función. Nunca `strftime` directo.

Autenticación

- Token JWT con `sub = username`, expiración 30 días
- Clave secreta almacenada en `.secret_key` (persiste entre reinicios)
- En producción, la clave se genera al primer arranque y se mantiene en el disco persistente de Render
- Cada endpoint protegido usa `Depends(get_current_user)` que valida el JWT y resuelve el objeto `User` de la DB

Frontend

Todo el React vive dentro de un `<script type="text/babel">` en `static/index.html`. Babel transpila el JSX en el browser al cargar la página. Es inusual en 2025, pero elimina completamente el build pipeline — útil para un proyecto personal.

Estructura de componentes

```
App
├─ AuthScreen      (login/registro)
├─ Header          (tasas + nav + usuario)
│   ├─ DashboardTab (cards resumen + gráficos + alertas budget + historial tasas)
│   ├─ TransactionsTab (tabla filtrable + export CSV + edición inline)
│   ├─ SavingsTab    (saldo USDT + metas de ahorro + historial USDT)
│   ├─ BudgetTab     (CRUD de presupuestos + barras de progreso)
│   └─ AITab         (chat con Gemini)
```

Estado global (App)

token, username	→ autenticación
rates, rateHistory	→ tasas de cambio
transactions	→ lista completa (cargada al inicio)
summary	→ totales + categorías + alertas budget
flow	→ datos del gráfico mensual
goals	→ metas de ahorro
budgets, budgetStatus	→ presupuestos + estado mes actual

Decisiones de diseño

¿Por qué un solo archivo Python?

Para un proyecto personal con un solo desarrollador, el overhead de módulos/paquetes no agrega valor. Cuando supere las ~1000 líneas o agregue workers de background, tendría sentido separarlo.

¿Por qué sin ORM migrations (Alembic)?

Las migraciones se hacen con ALTER TABLE ... ADD COLUMN directo al arrancar, protegido contra "duplicate column". Es suficiente para SQLite. En PostgreSQL con datos en producción, sí conviene migrar a Alembic antes de agregar columnas en producción con tráfico.

¿Por qué amount_ves desnormalizado?

Las queries de resumen, flujo mensual y alertas de presupuesto suman amount_ves directamente. Calcularlo en runtime requeriría JOINS o subconsultas con las tasas históricas, que ya no existen para transacciones viejas.

¿Por qué tags como CSV en una columna texto?

Simple y no requiere tabla intermedia. El tradeoff es que no podés hacer WHERE tag = 'x' con índice eficiente. Para uso personal con cientos de transacciones no importa; si escalara a miles conviene una tabla transaction_tags.